Object Oriented Analysis & Design

Project title: Green Justice

Course Module: Programme Construction CS1040

Group name: HackersShowDown

Name	Index Number
Mayoorthanan S	240426C
Danustan T	240086G
Abishan K	240017A
Jathursanan S	240261N
Pathuman S	240489U

➢ Introduction

1) **Vision:**
   - We create a user-friendly web application for decreasing the count of environmental law violations.

2) **Purpose:**
   - Reduce environmental violations and damage.
   - Enable quick and easy reporting of illegal activities.
   - Bridge the gap between citizens spotting issues and authorities taking action.
   - Organize reports in a structured, actionable, and transparent way.

3) **Target users:**
   I. People
      - Anyone who notices environmental violations.

   II. Environmental Authorities
      - Local councils and agencies responsible for investigating and resolving reported violations.

4) **Objectives**
   - People can report environmental violations easily and authorities can see real time complaints and manage them.

5) **System Overview**
   - Green Justice is a web-based complaint management system.
   - It has two major parts.
     I. User interface (For public)
     II. Authority Interface (Authority council officers)

6) **Objects**
   - User
   - Authority
   - Complaint
   - Violation
   - Locations
   - Evidence
   - Reminder
   - Dashboard
   - Status update
   - Authentication

7) **Features**

- **User side features**

I. Welcome page

- Language selection (English, Sinhala, Tamil)
- Report violation (Button)

II. Report violation process

Step 1: Violation type selection

- Illegal waste disposal
- Air pollution
- Water pollution
- Deforestation
- Noise pollution

Step 2: Upload the evidences

- Photo or video upload relevant to the violations.

Step 3: Share Location

- Integrated Map (Google Map API)

Step 4: Submit Report

Step 5: Thank You Page

III. Complaint status tracking

- Users can view status:
  - Not Viewed
  - In Progress
  - Resolved
- Users cannot modify status.
  - Making sense with security purpose

- **Authority side features**

I. Secure login page

- Only authority members can access (For protect from fake authorities)
- Mandatory credential verification

II. Dashboard

- Total complaints
- Resolved complaints
- Pending complaints
- Weekly statistics

III. Complaints Management
    - Sorted list
        - Highly reported and big violations and recently uploaded

IV. Reminder system
    - If complaint not addressed within 7 days, automatic reminder will send notification.

V. Status update options.
    - Not viewed.
    - In progress
    - Resolved.

VI. Fake report handling
    - Authority can delete reports

❖ **Other key features**

I. Report portal:
    - language selection → violation type → evidence upload → map-based location → confirmation.

II. Authority console:
    - login → complaint list with filters → detailed view → status updates/deletion.

III. Auto remainder for authorities because of non-viewed problems.

IV. Status updates for users for resolved or in progress or non-viewed.

➢ Functional Requirements

❖ For the User

- Language selection: Interface support at least two or more languages.
- Violation type selection: System should provide a list of environmental violations.
- Multimedia evidence upload: Allow to upload images or video files.
- Geolocation reporting: System should provide a map to pin the location.
- Report submission confirmation: System displays a “Thank You” message after successful submission.
- Status Tracking: Allow users to view the status of their report only in reading mode, whether “Not viewed” or “In Progress” or “Resolved”.

❖ For the Authority

- Secure Authentication: Mandatory login for verified authorities.
- Dashboard View: A sortable list of complaints based on Date Submitted Frequency of Reports and size of complaints.
- View Complaint Details: System should allow authorities to view violation type, uploaded medias, location and submission data.
- Automatic authority Suggestions: System automatically send the complaint to the authority by complaint region.
- Status Management: Able to update the complaint status.
- Automated Reminder System: Send a notification of reminder automatically, if a report is not accessed within 7 days.
- Real-Time Update Visibility: Any status updates made by authority should immediately appear on user side.

➢ Non-Functional Requirements

❖ Performance

- The map interface should load within 2 seconds.
- Media upload must begin immediately after file selection.
- Complaint submission must be completed within 3 seconds under normal conditions.
- Dashboard filtering must return results within 2 seconds.

- **Security**
  - Authority passwords must be stored using strong, secure hashing algorithms.
  - Access must be role based.
  - System must use HTTPS for secure communication.
  - Uploaded media must be securely stored.
  - Unauthorized access must be prevented.
  - Only authenticated users can delete reports.

- **Usability**
  - The interface must use clear icons and simple instructions.
  - Error messages should explain what went wrong in a way that's easy for any user to understand.
  - The design must work smoothly on mobile devices, since most users will access it from their phones.

- **Reliability**
  - Once a complaint is submitted, it must be saved.
  - The reminder system must in action accurately after 7 days of inactivity.
  - Every status update must be correctly recorded and clearly shown to the user.
  - The system must ensure 24/7 availability.
  - Regular database backups must be in place to protect records in case something goes wrong.

- **Scalability**
  - The system must be able to handle many users at the same time without slowing down.
  - The database must handle increasing volumes of complaints.
  - The system's architecture must be flexible enough to support new features in the future without requiring redesign.

- **Maintainability**
  - The system must follow modular design principles.
  - The application must follow Object-Oriented Programming principles
    I. Encapsulation
    II. Abstraction
    III. Inheritance
    IV. Polymorphism

- Code must be structured into separate modules
    I. User Module
    II. Authority Module
    III. Report Module
    IV. Notification Module
    V. Database Module
- System updates must not affect unrelated components.
- Proper documentation must be maintained.

➢ Object Oriented Analysis (OOA)

1) Introduction of OOA

- Before we build a software for a problem we must identify the important objects. This process called as OOA.
- The main goal of OOA is figure out what the system needs to do before coding. (Interactions between objects).

2) The core steps of OOA are

- Identify the Object and Classes.
- Attributes
- Methods
- Relationship

3) For our project Green Justice

○ Identification of core objects

I. User

❖ **Role**: Report the environmental violations.

❖ **Attributes:**
- Preferred language
- Reported complaints
- Status views

❖ **Methods:**
- Select language
- Submit complaint
- Upload evidences
- Pin location on map
- View complaint status

❖ **Relationships:**
- One-to-many relationship with complain

II. Authority

❖ **Role:** Allowed authorities and they can manage the problems.

- **Attributes:**
  - Authority Id
  - Name
  - Region
  - Assigned complaints
- **Methods:**
  - Login
  - Update complaint status
  - View department Info (complaint Id)
  - Trigger reminder (complaint Id)

- **Relationships:**
  - One-to-many with complaint

III. Complaint

- **Role:** Captures each violation report submitted by users.

- **Attributes:**
  - Complaint Id
  - Violation type
  - Media URL
  - Location coordinates
  - Report date
  - History

- **Methods:**
  - Set status
  - Compute reminder due date
  - Assign authorities by region.

- **Relationships:**
  - Many-to-one with user
  - One-to-many with status update

IV. Violations

- **Role:** Stored data of violation.

- **Attributes:**
  - Violation Id
  - Violation type

- **Methods:**
  - Check violation ID (User input)

- **Relationships:**
  - one to one with users.

V. Reminder

- **Role:** Tracks reminders for unresolved complaints older than a week.

- **Attributes:**
  - Reminder Id
  - Complaint Id
  - Scheduled date
  - Sent status
  - Recipient authority

- **Methods:**
  - Check eligibility (complaint)
  - Dispatch reminder
  - Mark as sent

- **Relationships:**
  - One-to-one with complaint

VI. Status update

- **Role:** Updates the status of problems.

- **Attributes:**
  - Complaint Id
  - Updated by authority Id
  - Time stamp
  - Status
  - Description

- **Methods:**
  - Log update
  - Get timeline

- **Relationships:**
  - Many - to - one with complaint

VII. Dashboard

- **Role:** Represents the admin/authority dashboard with real-time stats and filtering tools.

- **Attributes:**
  - Active filters
  - Complaint list
  - Sort mode
  - Stats (counts by status, severity, etc.)

- **Methods:**
  - Sort complaints
  - Filter complaints
  - Show real time updates
  - Display reminder alerts

- **Relationships:**
  - Displays aggregated complaint, status update, reminder

VIII. Authentication

- **Role:** Manages secure access for authorities.

- **Attributes:**
  - Authority id
  - Last active

- **Methods:**
  - Validate credentials
  - Generate session
  - Logout

- **Relationships:**
  - One-to-one with authority

IX. Evidence

- **Role:** Stores and validates media attached with complaints.

- **Attributes:**
  - Evidence Id
  - Complaint Id
  - File type
  - File URL
  - Uploaded location
  - Time stamp

- **Methods:**
  - Upload (file)
  - Check location
  - Delete

- **Relationships:**
  - Many-to-one with complaint

X. Locations

- **Role:** Represent the geo-spatial context of violation.

- **Attributes:**
  - Location Id
  - Longitude
  - Land mark
  - District

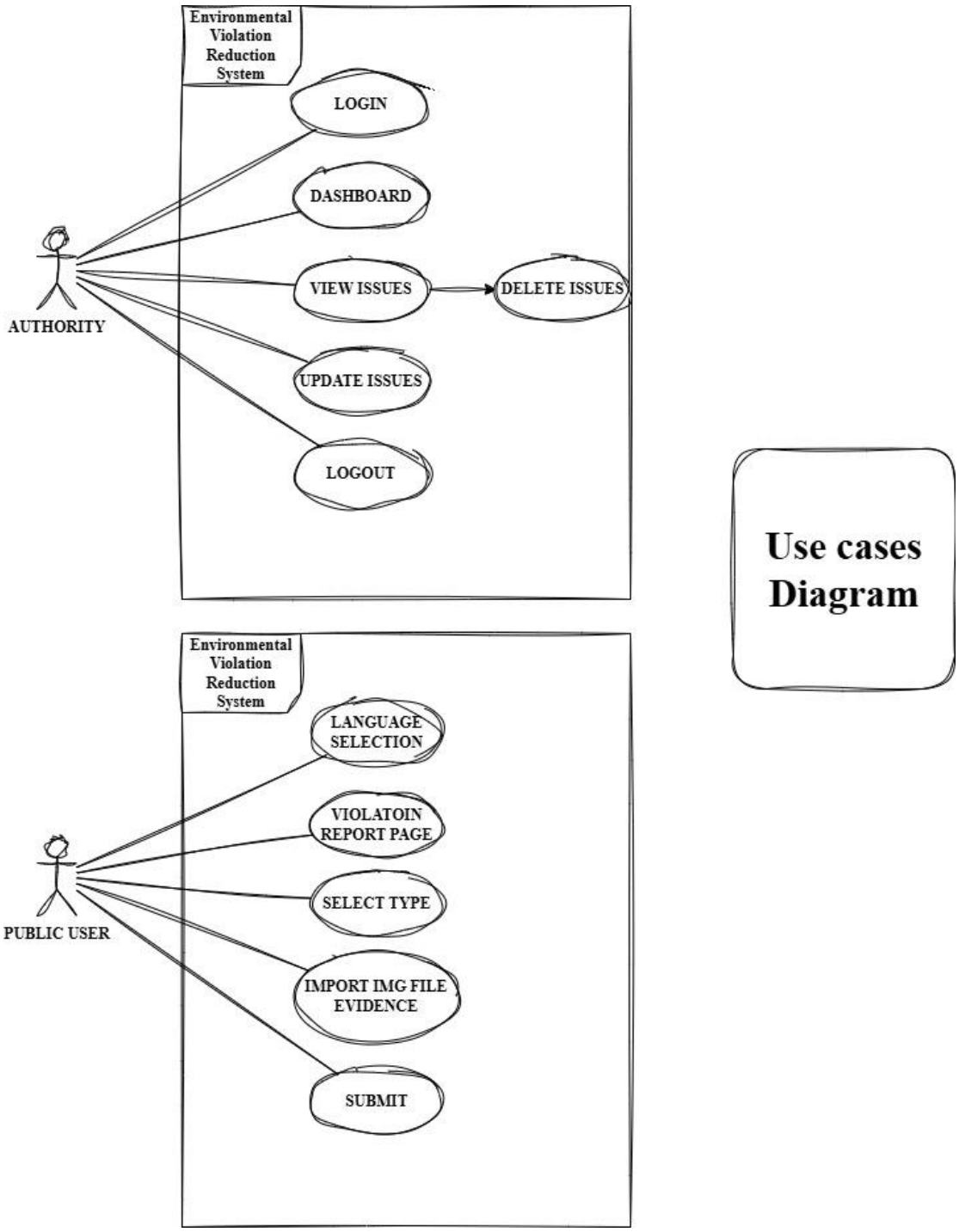
- **Methods:**
  - Set coordinates
  - Reverse geo code
  - export geo Json (provides data for map-based dashboard)

- **Relationships:**
  - One-to-one with complaint

4) **Diagrams**

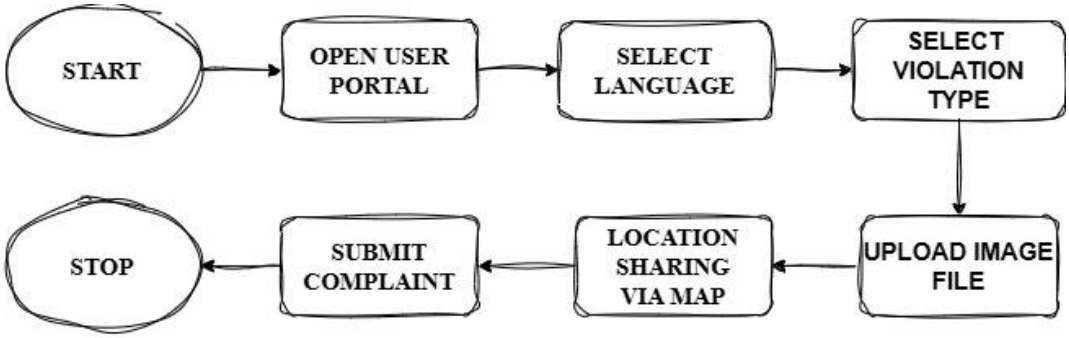
I. Use Case Diagram

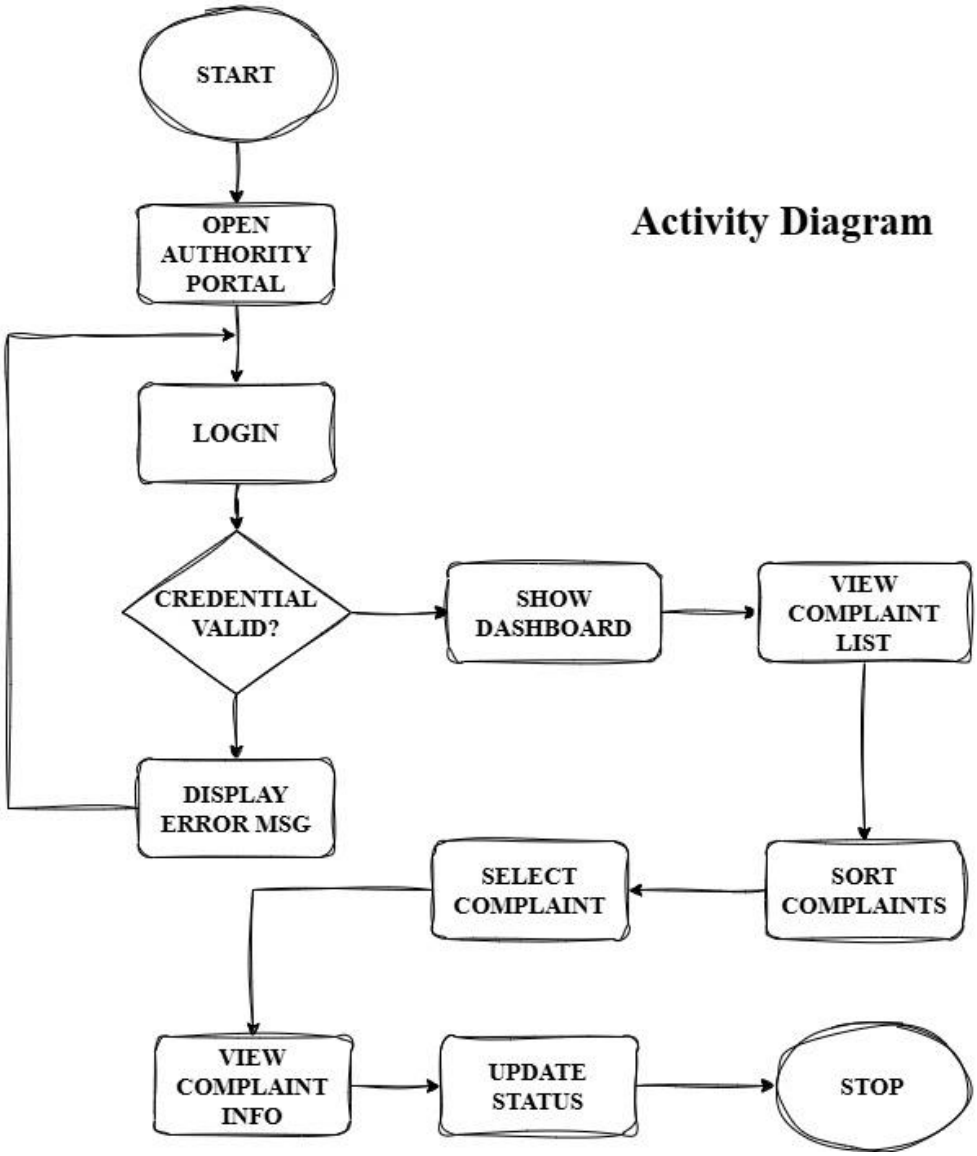
- This will identify the interaction between external actors and the systems.



II. Activity Diagram

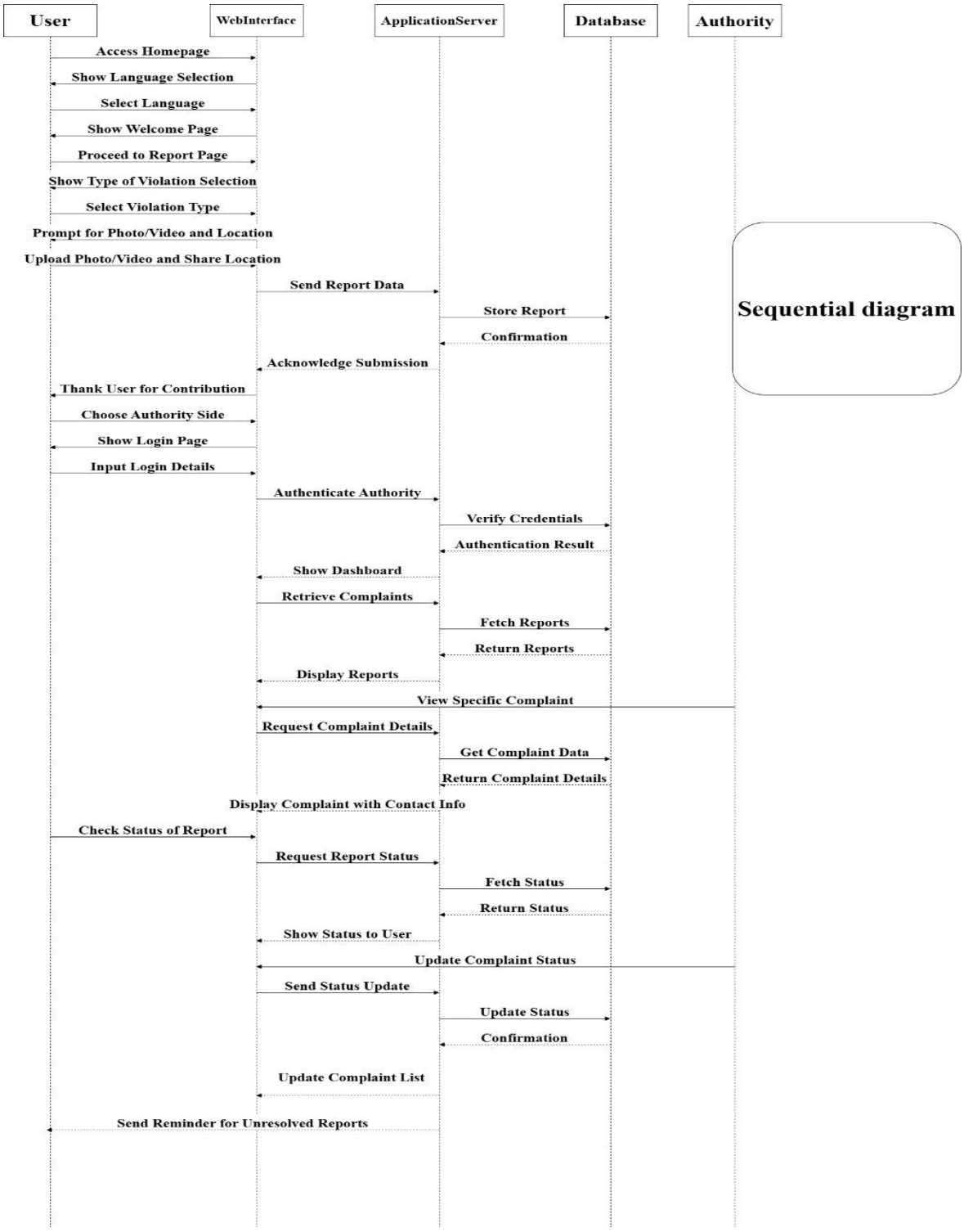
- This is a UML flowchart that shows the step-by-step actions and decision points in a process, highlighting how the system moves from one activity to the next.





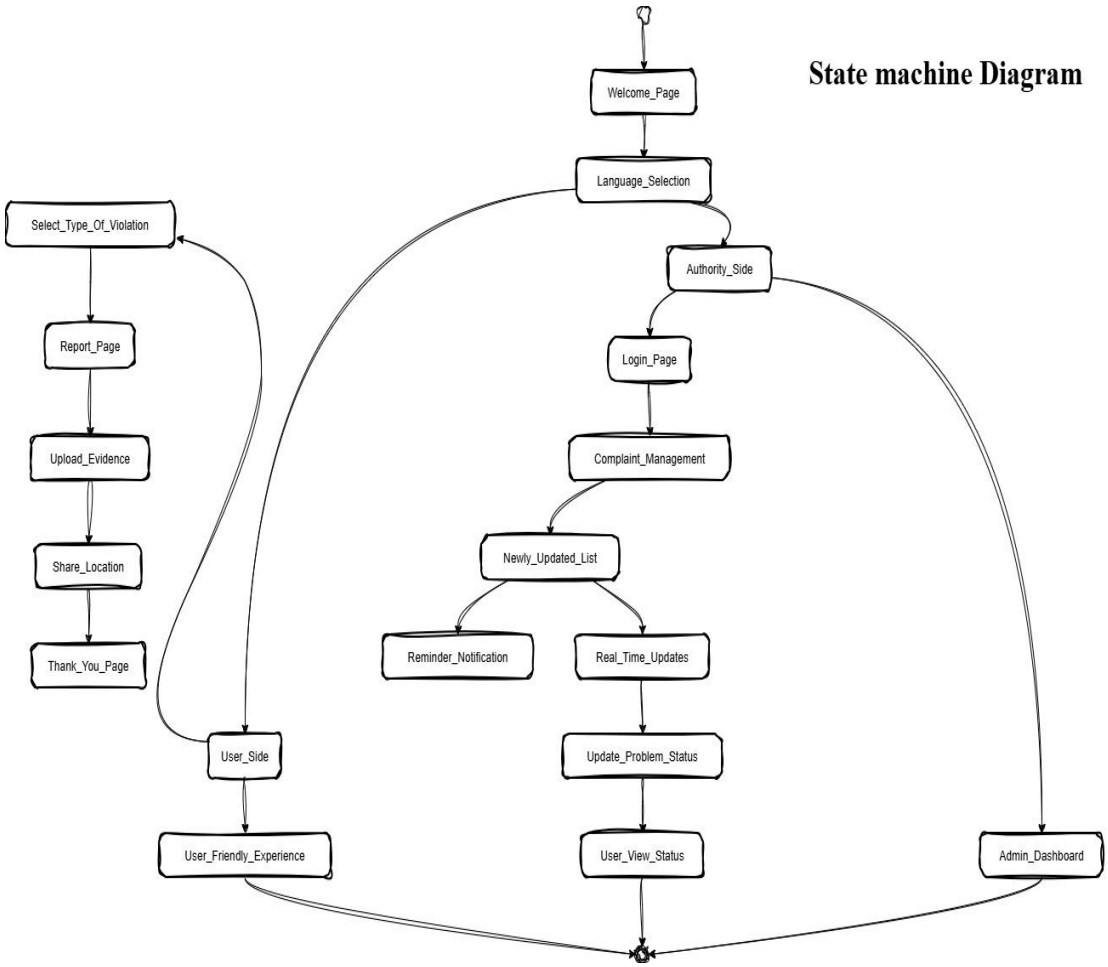
III. Sequence diagram

- This is a UML illustration that shows how objects interact over time by exchanging messages.



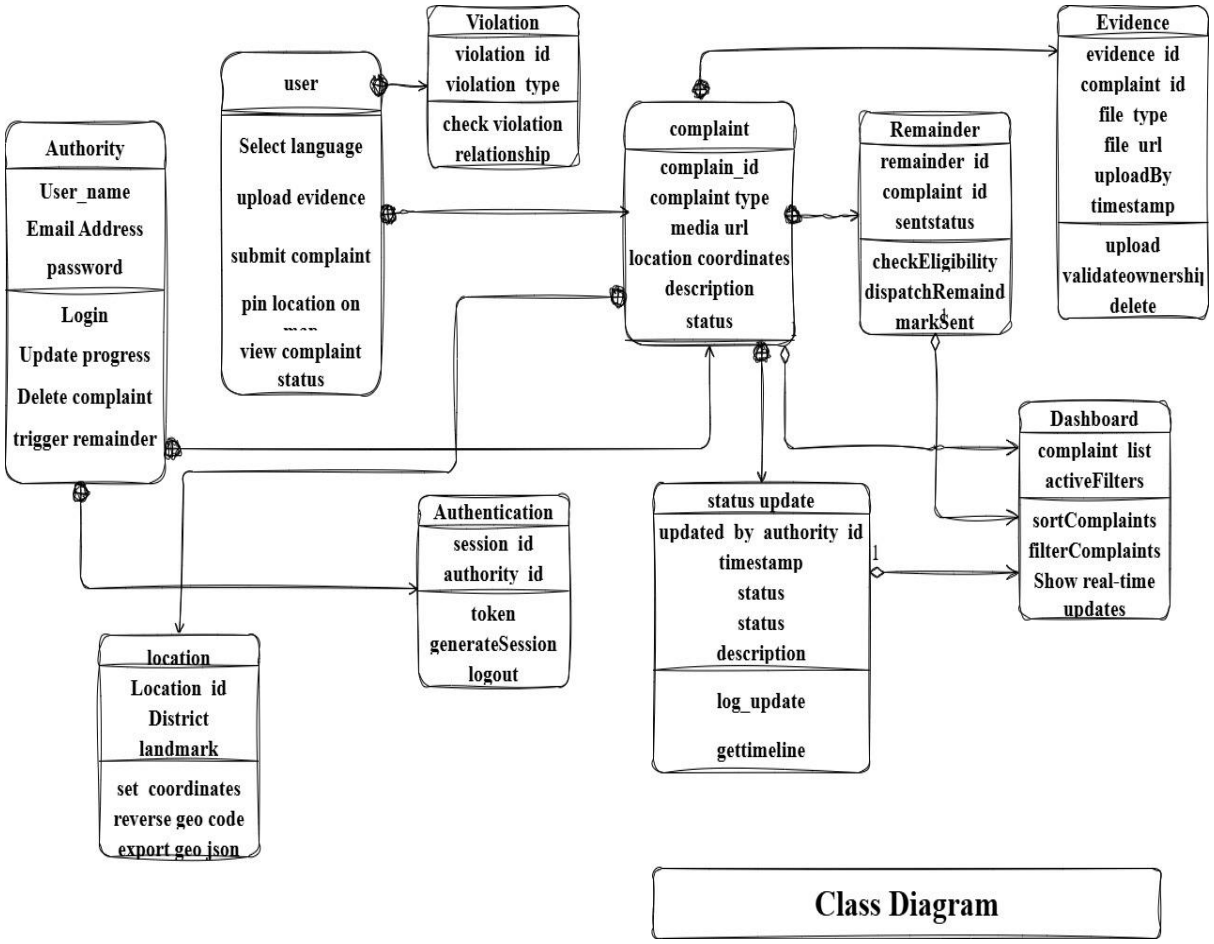
IV. State Machine Diagram

- This diagram shows the different states an object can be in and how it transitions between those states in response to events.



V. Class interaction Diagram

- A class interaction diagram shows how class instances interact by sending messages to each other within a specific scenario. It focuses on which objects collaborate and the order of their interactions, helping you understand how responsibilities are distributed across classes.



Class Diagram

➢ Tools used

- The UML diagrams were drawn by manually using with Draw io. The final report was prepared using MS Office Word by manually.

- Draw io URL is draw.io

➢ References

1) GeeksforGeeks – UML references - You searched for UML Diagrams - GeeksforGeeks

2) Booch, G. (2007). *Object-oriented analysis and design with applications* (3rd ed., pp. 221–260). Addison-Wesley. - Object.Oriented.Analysis.and.Design.with.Applications.3rd.Edition.by.Booch.pdf

3) Draw io URL is https://www.drawio.com/

4) The Central Environmental Authority (CEA) manages natural resources and environmental conservation in Sri Lanka - Central Environmental Authority (CEA)

5) Wikipedia - Environmental crime - Wikipedia

6) YouTube - Object Oriented Design - YouTube